

UNIVERSIDADE KIMPA VITA

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

# RELATÓRIO TÉCNICO DE LÓGICA DE PROGRAMAÇÃO

TEMA:

## SISTEMA DE GESTÃO DE DADOS DE ESTUDANTES

Autores:

**Heraclito Kambaiala Ferreira Kussei**  
**João António Massamba Luzembo**  
**Abel C. Pedro Malo**

**Sala:** Auditório nº 2  
**Turma:** TL101  
**Período:** Manhã  
**Classe:** 1.º Ano

Orientadores:

**Moyo Kanivengidio**  
**Paulo Miezi João**

**Uíge – Angola**  
**2026**

## Introdução

A linguagem C é uma das linguagens de programação mais utilizadas no desenvolvimento de software devido à sua eficiência, simplicidade e capacidade de controlo sobre os recursos do computador. No âmbito académico, a aprendizagem desta linguagem permite compreender conceitos fundamentais como estruturas de dados, manipulação de ficheiros, algoritmos de ordenação e algoritmos de pesquisa.

O presente trabalho apresenta um Sistema de Gestão de Estudantes desenvolvido em linguagem C. O programa tem como objetivo permitir o carregamento, organização, pesquisa e armazenamento de informações de estudantes através de ficheiros de texto e ficheiros binários. Além disso, o sistema implementa diferentes algoritmos de ordenação e pesquisa, permitindo ao utilizador manipular os dados de forma eficiente através de um menu interativo.

Este relatório descreve detalhadamente o funcionamento de cada módulo do programa, explicando a sua estrutura e a forma como os diferentes componentes interagem entre si.

## Desenvolvimento

### 1. Estrutura de Dados

O programa utiliza uma estrutura denominada **Estudante**, criada através da palavra-chave struct.

```
typedef struct {  
    int matricula;  
    char nome[50];  
    float nota;  
} Estudante;
```

Esta estrutura permite armazenar as seguintes informações:

- Matrícula do estudante;
- Nome do estudante;
- Nota do estudante.

A utilização de estruturas facilita o agrupamento de informações relacionadas num único registo.

---

### 2. Manipulação de Ficheiros

O sistema possui funções responsáveis pela leitura e gravação de dados.

#### 2.1 Leitura de Fiche

iro de Texto

A função lerFicheiroTexto() abre um ficheiro de texto e lê os dados dos estudantes.

```
fscanf(ficheiro, "%d,%[^,],%f\n",
        &estudantes[i].matricula,
        estudantes[i].nome,
        &estudantes[i].nota);
```

O formato esperado no ficheiro é:

```
1001,Joao,15.5
1002,Maria,18.0
1003,Carlos,12.5
```

Após a leitura, os dados são armazenados num vetor de estruturas.

## 2.2 Gravação de Ficheiros

A função gravarFicheiro() permite guardar os dados em:

### **Ficheiro de Texto (.txt)**

```
fprintf(ficheiro, "%d,%s,%.2f\n",
        estudantes[i].matricula,
        estudantes[i].nome,
        estudantes[i].nota);
```

### **Ficheiro Binário (.bin)**

```
fwrite(estudantes,
        sizeof(Estudante),
        n,
        ficheiro);
```

Os ficheiros binários ocupam menos espaço e permitem uma leitura mais rápida dos dados.

---

## 3. Algoritmos de Ordenação

O sistema implementa três algoritmos clássicos de ordenação.

### 3.1 Bubble Sort

Função:

`bubbleSortMatricula()`

Objetivo:

Ordenar os estudantes pela matrícula em ordem crescente.

Funcionamento:

- Compara elementos adjacentes;
- Troca os elementos quando estão fora de ordem;
- Repete o processo até que toda a lista esteja ordenada.

Exemplo:

Antes:

1005  
1001  
1003

Depois:

1001  
1003  
1005

---

### 3.2 Selection Sort

Função:

`selectionSortNome()`

Objetivo:

Ordenar os estudantes alfabeticamente pelo nome.

Funcionamento:

- Procura o menor elemento da parte não ordenada;
- Coloca-o na posição correta;
- Repete o processo até ao fim da lista.

Exemplo:

Antes:

Carlos  
Ana  
Bruno

Depois:

Ana  
Bruno  
Carlos

---

### 3.3 Insertion Sort

Função:

insertionSortNota()

Objetivo:

Ordenar os estudantes pela nota em ordem decrescente.

Funcionamento:

- Insere cada elemento na posição correta da parte já ordenada;
- Mantém a lista organizada a cada iteração.

Exemplo:

Antes:

12  
18  
15

Depois:

18  
15  
12

---

## 4. Algoritmos de Pesquisa

O sistema permite localizar estudantes através de diferentes métodos.

### 4.1 Pesquisa Linear por Nome

Função:

`pesquisaLinearNome()`

Percorre toda a lista comparando os nomes até encontrar o estudante procurado.

Vantagem:

- Simples de implementar;
  - Não exige ordenação prévia.
- 

### 4.2 Pesquisa Linear por Matrícula

Função:

`pesquisaLinearMatricula()`

Percorre a lista verificando cada matrícula.

Vantagem:

- Fácil implementação;
  - Funciona mesmo com dados não ordenados.
- 

### 4.3 Pesquisa Binária por Matrícula

Função:

`pesquisaBinariaMatricula()`

Este método só pode ser utilizado quando a lista estiver previamente ordenada por matrícula.

Funcionamento:

1. Localiza o elemento central.

2. Compara com a matrícula procurada.
3. Elimina metade da lista.
4. Repete até encontrar o elemento.

Vantagem:

- Muito mais rápida do que a pesquisa linear.

Complexidade:

- Pesquisa Linear:  $O(n)$
  - Pesquisa Binária:  $O(\log n)$
- 

## 5. Funções de Interface

Exibir Lista

Função:

`exibirLista()`

Apresenta todos os estudantes em formato tabular.

Exemplo:

| Matricula | Nome | Nota |
|-----------|------|------|
| 1001      | Ana  | 18.0 |
| 1002      | João | 16.5 |

Exibir Estudante

Função:

`exibirEstudante()`

Mostra apenas os dados do estudante encontrado numa pesquisa.

---

## 6. Programa Principal

A função `main()` controla todo o funcionamento do sistema.



O menu principal apresenta as seguintes opções:

1. Ler ficheiro de texto;
2. Ordenar lista;
3. Pesquisar estudante;
4. Gravar ficheiro;
5. Exibir lista;
6. Sair.

O programa utiliza uma estrutura de repetição:

```
do {  
    ...  
} while(opcao != 0);
```

Isto garante que o menu permaneça ativo até que o utilizador escolha a opção de saída.

Também é utilizada uma variável de controlo:

`listaEstaOrdenadaPorMatricula`

Esta variável verifica se a lista já foi ordenada por matrícula antes de permitir a utilização da pesquisa binária.

Desta forma evita-se a execução incorreta do algoritmo.

## Conclusão

O Sistema de Gestão de Estudantes desenvolvido em linguagem C demonstra a aplicação prática de vários conceitos fundamentais da programação, incluindo estruturas de dados, manipulação de ficheiros, algoritmos de ordenação e algoritmos de pesquisa.

O programa permite carregar dados a partir de ficheiros, organizar as informações segundo diferentes critérios, localizar estudantes através de pesquisas eficientes e armazenar novamente os dados em ficheiros de texto ou binários.

A implementação dos algoritmos Bubble Sort, Selection Sort, Insertion Sort, Pesquisa Linear e Pesquisa Binária proporciona uma compreensão clara das diferentes técnicas de processamento de dados estudadas na disciplina de Programação em C.

Conclui-se que o sistema cumpre os objetivos propostos, oferecendo uma solução funcional para a gestão de estudantes e servindo como excelente exemplo académico para o estudo de algoritmos e estruturas de dados.